

13b — Komplexität verstehen

Wie schnell wächst der Aufwand? Big-O als Sprache

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Warum reicht “läuft schnell auf meinem Rechner” nicht?
2. Big-O-Notation – die Sprache der Komplexität
3. Die sechs wichtigsten Klassen: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$
4. Komplexität *zählen* an konkreten Funktionen
5. Speicherkomplexität – der zweite Aufwand

Lernziel: Sie können die Komplexitätsklasse einer Funktion am Code ablesen – und vorhersagen, wann sie kippt.

Wie wir heute arbeiten: *Live-Demo* zeigen wir im Terminal, *Sofort ausprobieren* in Notebook 13b.

Motivation: zwei Wege, ein Ergebnis

Variante A: Schleife

```
def summe_a(n):  
    summe = 0  
    for i in range(1, n + 1):  
        summe += i  
    return summe
```

Variante B: Formel

```
def summe_b(n):  
    return n * (n + 1) // 2
```

n	Variante A	Variante B
100	100 Schritte	1 Schritt
1 000 000	1 000 000 Schritte	1 Schritt

Big-O-Notation - die Idee

- **O(...)** beschreibt das *Wachstumsverhalten*, **nicht** die genaue Zeit
- Konstanten und niedrige Terme werden weggekürzt:
 - $3n^2 + 5n + 7$ ist **O(n²)**
- Wir betrachten in der Regel den **Worst Case** (schlechtester Fall)

Lesart

“Diese Funktion ist O(n).” = “Wenn die Eingabe doppelt so groß wird, dauert die Funktion *etwa* doppelt so lange.”

Die sechs wichtigsten Klassen

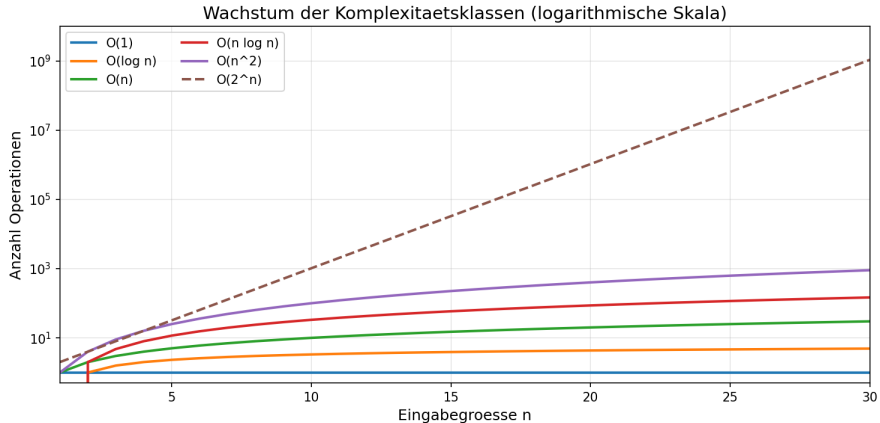


Abbildung 1: Wachstumskurven der Komplexitätsklassen

O(1) und O(n) im Vergleich

O(1) -- konstante Zeit

```
def erstes_element(liste):  
    return liste[0]
```

O(n) -- lineare Zeit

```
def summe_liste(liste):  
    summe = 0  
    for x in liste:  
        summe += x  
    return summe
```

- `erstes_element`: egal wie lang die Liste, **immer ein Schritt**
- `summe_liste`: doppelt so lange Liste → doppelt so viele Additionen

$O(\log n)$ - die Halbierungs-Idee

Bei jedem Schritt fällt die Hälfte des Problems weg.

$n = 1\ 000 \rightarrow$ ca. 10 Schritte

$n = 10\ 000 \rightarrow$ ca. 14 Schritte

$n = 100\ 000 \rightarrow$ ca. 17 Schritte

Beispiel: Telefonbuch in der Mitte aufschlagen, dann nur noch in der relevanten Hälfte weiter suchen.

Vorschau: Die **Binärsuche** in NB 14b ist genau so – $O(\log n)$ statt $O(n)$.

$O(n^2)$ - verschachtelte Schleifen

```
def hat_duplikat(liste):  
    for i in range(len(liste)):  
        for j in range(i + 1, len(liste)):  
            if liste[i] == liste[j]:  
                return True  
    return False
```

- Äußere Schleife: n Mal
- Innere Schleife: bis zu n Mal
- Insgesamt: $n \times n = n^2$ Vergleiche im Worst Case

Faustregel: Verschachtelte Schleifen über dieselbe Liste sind ein Warnsignal
– Wachstum kippt schnell.

Visualisierung: Wachstum als ASCII-Balken

```
n = 1      | #  
n = 4      | ####  
n = 16     | #####  
n = 64     | ##### ...
```

Demo zeigt n vs. n^2 als Balken – man *sieht*, wie n^2 explodiert.

Visualisierungsstufe 2: ASCII-Balken. Wir nehmen genau diese Funktion `print_balken(...)` in 14a wieder her, um Sortier-Schritte sichtbar zu machen.

► **Live-Demo** – `02_komplexitaetstabelle.py` und `03_ascii_balken.py`

Tabellen-Vergleich

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$
-----+-----+-----+-----+-----				
10	1	3	10	100
100	1	6	100	10 000
1000	1	9	1000	1 000 000

- $O(n^2)$ wächst zehntausendfach schneller als $O(n)$ bei $n=1000$
- $O(\log n)$ bleibt fast konstant – darum so wertvoll für Suche

 **Sofort ausprobieren** – Notebook 13b, Abschlussübung 1: Komplexität von `finde_minimum` bestimmen

Speicherkomplexität (in einem Satz)

Was tun wir?	Speicher
Eine Variable pro Schritt (summe, i)	$O(1)$
Neue Liste mit n Elementen erzeugt	$O(n)$
Matrix $n \times n$ erzeugt	$O(n^2)$

$O(1)$ Speicher

```
def max_finden(liste):  
    m = liste[0]  
    for x in liste:  
        if x > m: m = x  
    return m
```

$O(n)$ Speicher

```
def quadrate(n):  
    ergebnis = []  
    for i in range(n):  
        ergebnis.append(i*i)  
    return ergebnis
```

► **Live-Demo** - 04_speicher_iterativ_vs_rekursiv.py

Cheat Card: Komplexitätsklassen

Klasse	“Wenn n verdoppelt wird ...”	Beispiel
$O(1)$	bleibt gleich	<code>liste[i]</code>
$O(\log n)$	+1 Schritt	Binärsuche
$O(n)$	dauert doppelt	Liste durchlaufen
$O(n \log n)$	etwas mehr als doppelt	<code>Pythons sorted()</code>
$O(n^2)$	viermal so lange	Bubble Sort
$O(2^n)$	quadrat so lange	naive rekursive Fibonacci

Faustregel: Verschachtelte Schleifen $\rightarrow O(n^2)$. Halbieren in jedem Schritt $\rightarrow O(\log n)$.

Ausblick: 14a - drei Sortieralgorithmen

- Bubble Sort und Insertion Sort: **$O(n^2)$**
- Quicksort: durchschnittlich **$O(n \log n)$** , im Worst Case $O(n^2)$
- Pythons `sorted()` / `list.sort()`: **TimSort**, $O(n \log n)$

Wir bauen auf den ASCII-Balken von heute auf – sehen Sortier-Schritte als visuelle Spur.

Vorschau: Bubble Sort

```
for i in range(n):  
    for j in range(n - i - 1):  
        if liste[j] > liste[j+1]:  
            liste[j], liste[j+1] = liste[j+1], liste[j]
```

- ✓ Big-O-Notation als Sprache verstanden
- ✓ Die sechs wichtigsten Klassen unterschieden
- ✓ Komplexität durch Operationen-Zählen ermittelt
- ✓ ASCII-Balken als Visualisierungs-Werkzeug genutzt
- ✓ Zeit- und Speicherkomplexität getrennt betrachtet

 Zur Vertiefung im Notebook 13b:

- Angeleitete Übungen 3.1, 3.2, 4.1, 4.2, 5.1, 5.2 zu Komplexitätsklassen
- Abschlussübungen 1-4: `finde_minimum`, `enthält_element`, `zähle_vorkommen`, `matrix_transponieren`